# Hyper BPFLoader:Android 平台上 eBPF 动态加载管控实践

The 4th eBPF Developer Conference (eBPFDC 2026)

小米科技有限责任公司 | 内核技术部
高翔

陕西 · 西安　　2026年4月18日

# 目录

作者介绍

现状痛点

需求目标

方案设计

演进思考

# 作者介绍

eBPF 技术爱好者，主要专注 Android 系统稳定性方向，有多年底层优化与内核可观测性经验。长期探索 eBPF 技术在移动端的工程化落地，乐于在系统性能分析、内核追踪与可观测性建设等领域持续实践和分享。

# 现状

- **启动阶段一次性加载**
  bpfloader 仅在系统启动时运行一次，所有 BPF 程序必须在此时全部加载完成
- **运行时无法变更**
  系统启动后，无法追加、更新或重新加载任何 BPF 程序

# 痛点

这种一次性加载策略虽在一定程度上提升了安全性，但随着 eBPF 程序数量不断增长，所有程序在开机阶段集中加载会显著**增加内存占用**并**延长启动时间**，对设备的开机体验和性能指标造成负面影响。

**Android eBPF 内存占用分析**

BPF Programs & Maps 运行时内核内存估算

| 来源 | MAP 内存 | PROG 内存 | 合计 | 占比 |
|---|---|---|---|---|
| Google AOSP | 7.5 MB | 0.3 MB | 7.8 MB | 34% |
| 某 OEM 厂商 | 15.0 MB | 0.1 MB | 15.1 MB | 66% |
| 全部 BPF | 22.5 MB | 0.4 MB | 22.9 MB | 100% |

**Android eBPF 加载时间分析**

开机 BPF Programs 加载耗时对比

| 来源 | 加载器 | 耗时 | PROGS 数 | 占比 |
|---|---|---|---|---|
| Google AOSP | netbpfload | 884 ms | 68 | 78% |
| 某 OEM 厂商 | hyper_bpfloader | 245 ms | 34 | 22% |
| 全部 BPF | 串行加载 | 1,134 ms | 102 | 100% |

# 需求目标

## 核心能力需求

- 支持 eBPF 程序**按需动态加载/卸载**，无需开机一次性全量加载
- 兼容 Libbpf 格式 BPF OBJ，提升开发灵活性与生态兼容性

## 安全管控需求

- 基于 ELF 内嵌签名机制，对未授权 BPF 程序进行检测与拦截，防止解锁设备与供应链场景下的篡改与替换
- 完善统计与审计信息，覆盖加载、卸载、拦截、异常等关键维度
- 建立规范的评审与管控流程，保障设备性能与系统稳定性

# 方案设计 – 整体框架

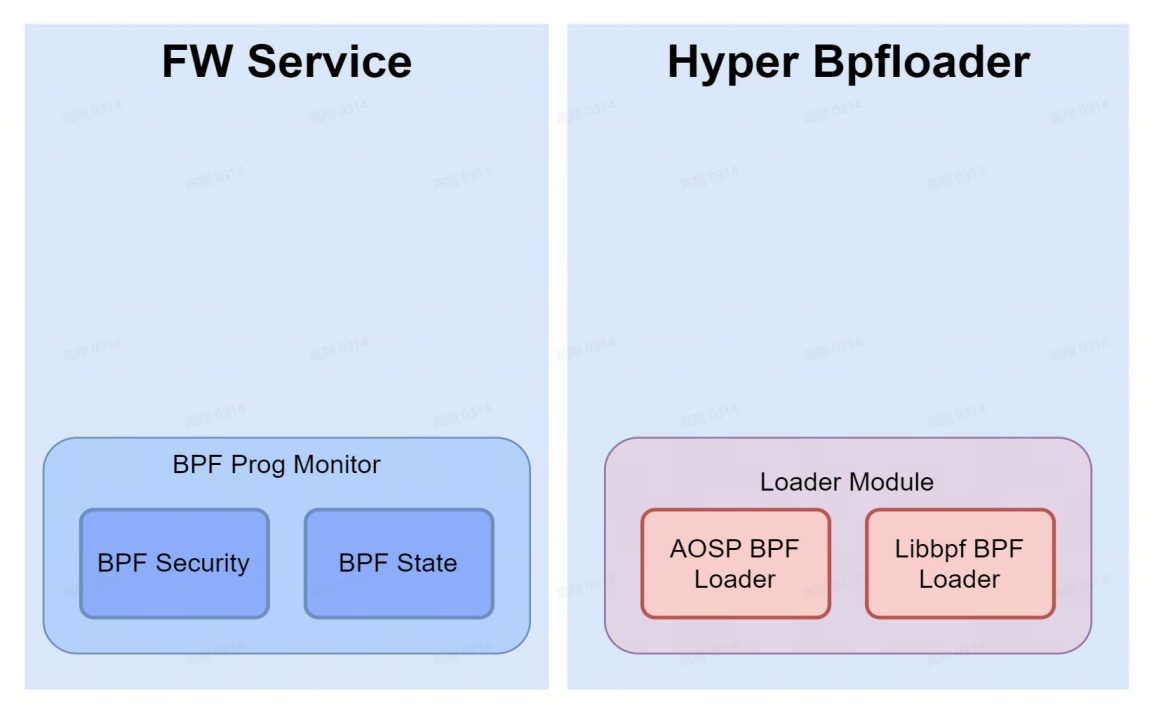
分为两个核心模块：

**Hyper Bpfloader**

- 解析 ELF 文件，创建 Map 并加载 Prog
- 支持 AOSP 与 Libbpf 双加载模式

**FW Service**

- 加载前验证签名与合规校验
- 运行状态监控与全链路审计



# 方案设计 - Hyper Bpfloader

在原生 bpfloader 基础上增加 Hyper Bpfloader，具备以下核心能力：

- **动态采集系统数据**：仅使能平台所需的 eBPF 程序，灵活适配不同项目需求与硬件配置
- **动态生成 eBPF 程序**：通过程序模板在系统运行时动态生成新程序，满足业务扩展需求
- **动态下发 eBPF 程序**：云端维护程序集，按需动态推送至设备，实现热更新与热修复能力

# 动态采集 — 按需使能 eBPF 程序

通过云端下发采集策略，设备端根据设备类型与场景按需使能对应的 BPF 程序，实现灵活的数据采集能力。

- 云端根据设备类型判定所需的 BPF 采集策略
- 设备端按策略加载全部、必要或测试 BPF 程序
- 通过 HyperBpfloader Socket 接口按需加载

# 动态生成 — 通过模板产生新 BPF 程序

基于预定义的 BPF 模板，云端根据采集需求动态生成新的 BPF 程序，无需重新编译固件即可扩展采集能力。

- 云端维护 BPF 模板库，按需填充参数生成目标程序
- 生成后自动编译、签名并下发至设备端
- 支持快速响应新增采集需求，无需 OTA 升级

# 动态下发 — 云端热更新

## 🔄 BPF 热更新实例 — 新增 GPU 频率监控能力

线上设备通过热更新获得新的 BPF 监控程序，无需 OTA，无需重启

**更新前 — 设备已有的 BPF 程序列表**

| | | |
|---|---|---|
| sched_latency.o | v1.3 | 调度延迟监控 |
| mem_pressure.o | v2.1 | 内存压力监控 |
| thermal_monitor.o | v1.0 | 温控状态追踪 |
| battery_drain.o | v1.5 | 耗电异常检测 |

⚠ 缺少 GPU 频率监控能力 — 无法分析游戏场景下的 GPU 调频策略问题

→ 热更新

**更新后 — 设备 BPF 程序列表（新增 GPU 监控）**

| | | |
|---|---|---|
| sched_latency.o | v1.3 | 调度延迟监控 |
| mem_pressure.o | v2.1 | 内存压力监控 |
| thermal_monitor.o | v1.0 | 温控状态追踪 |
| battery_drain.o | v1.5 | 耗电异常检测 |
| gpu_freq_monitor.o | v1.0 NEW | GPU 频率监控 |

**gpu_freq_monitor.c（新增热更新程序）** NEW

```
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 256 * 1024);
} gpu_events SEC(".maps");

SEC("tracepoint/power/gpu_frequency")
int trace_gpu_freq(struct trace_event_raw_gpu_freq *ctx) {
    struct gpu_event *e = bpf_ringbuf_reserve(&gpu_events, sizeof(*e), 0);
    if (!e) return 0;
    e->freq      = ctx->state;
    e->gpu_id    = ctx->gpu_id;
    e->ts        = bpf_ktime_get_ns();
    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

**🎮 GPU 频率监控能力说明**

| | |
|---|---|
| 监控点 | tracepoint/power/gpu_frequency |
| 采集数据 | GPU 频率、GPU ID、时间戳 |
| 数据通道 | BPF RingBuf → 用户态分析服务 |
| 应用场景 | 游戏卡顿分析、GPU 调频策略优化 |
| 更新方式 | 云端下发，Bpfloader 热加载 |

## 📡 BPF 热更新流程 — 云端下发新 BPF 程序到设备

| Day 1 | Day 2 | Day 3 | Day 4 | Day 5-6 | Day 7-8 |
|---|---|---|---|---|---|
| 需求确认 | 开发完成 | 编译签名 | 上传云端 | 灰度验证 | 全量推送 |

**☁ 云端**

- 📝 创建更新任务 — 配置目标设备/版本
- 📦 上传 .o 文件 — gpu_freq_monitor.o
- 🔍 安全审核 — 自动化 + 人工审核
- 📋 配置推送策略 — 灰度比例 / 设备条件
- 📡 下发指令 — 推送到目标设备

**📱 设备端**

- 📥 接收 .o — TLS 下载 + 完整性校验
- ✅ 验签 — .bpf_sig ECDSA 验证
- 📖 解析加载 — ELF 解析 + CO-RE 重定位
- 📌 Attach — 挂载 tracepoint + Pin Map
- 📊 上报结果 — 加载状态回传云端

**⚙ 内核**

- 🔧 bpf() 系统调用 — 加载 BPF 字节码到内核
- 🔗 Verifier 验证 — 安全检查 + 复杂度校验
- ⚡ JIT 编译 & 运行 — 转为本地机器码执行
- 📤 RingBuf 输出 — 事件数据送回用户态

**📊 关键指标**

| | | |
|---|---|---|
| 下发耗时 | < 5 分钟 | 从云端触发到设备加载完成 |
| 覆盖范围 | 百万级设备 | 按条件精准圈选目标设备 |
| 生效方式 | 即时生效 | 无需重启，无需 OTA |
| 安全保障 | 三重校验 | TLS + SHA256 + 签名验证 |

**⚡ 热更新 vs 传统 OTA**

| 对比维度 | 🐢 传统 OTA | ⚡ BPF 热更新 |
|---|---|---|
| 下发周期 | 2 ~ 4 周 | 数小时 |
| 用户操作 | 需要下载安装重启 | 无感知 |
| 影响范围 | 全系统更新 | 仅目标 BPF 程序 |
| 回滚能力 | 需再次 OTA | 远程即时回滚 |
| 覆盖精度 | 按机型批量 | 按设备条件精准圈选 |

# 动态下发 — 云端热修复

## 🔥 BPF 热修复实例 — 修复 sched_latency.o 调度延迟误报 Bug

线上发现 BPF 程序误报高延迟告警，通过热修复在数小时内修复，无需 OTA

**BUG VERSION** sched_latency.o (当前线上版本 v1.2)

```
1  SEC("tracepoint/sched/sched_switch")
2  int trace_sched_latency(struct trace_event_raw_sched_switch *ctx) {
3      u64 delta = bpf_ktime_get_ns() - start_ts;

5      // BUG: 阈值写死 1ms，在低端机上频繁误报
6      if (delta > 1000000) {          // ← 问题在这里
7          report_latency(ctx, delta);
8      }
9      return 0;
10 }
```

⟶ 热修复

**HOTFIX** sched_latency.o (热修复版本 v1.3)

```
1  SEC("tracepoint/sched/sched_switch")
2  int trace_sched_latency(struct trace_event_raw_sched_switch *ctx) {
3      u64 delta = bpf_ktime_get_ns() - start_ts;

5      // FIX: 从 Map 读取阈值，支持按机型动态配置
6      u64 *threshold = bpf_map_lookup_elem(&config, &key);
7      u64 thr = threshold ? *threshold : 5000000; // 默认5ms
8      if (delta > thr) {
9          report_latency(ctx, delta);
10     }
11     return 0;
12 }
```

⚠️ **线上影响**

低端机每小时误报 200+ 次高延迟告警，导致用户收到大量卡顿提示

✅ **修复效果**

误报率下降 99%，低端机阈值自动设为 5ms，高端机保持 2ms，可通过 Map 远程调整

### 📋 修复要点总结

- 🐛 **问题根因**
  延迟阈值硬编码 1ms
  低端机 CPU 频率低，正常调度>1ms
- 🔧 **修复方案**
  阈值改为从 BPF Map 动态读取
  默认 5ms，可按机型远程配置
- 📦 **修复方式**
  云端下发新 .o 替换旧版本
  无需 OTA，Hyper Bpfloader 热加载
- ⏱ **修复时效**
  发现→修复→全量: 6 小时
  传统 OTA 方式需要 2-4 周

## ⚡ BPF 热修复流程 — 从发现问题到全量修复

| T+0h | T+1h | T+2h | T+3h | T+6h |
|---|---|---|---|---|
| 发现问题 | 开发修复 | 编译签名 | 灰度验证 | 全量推送 |

**问题发现**
- 📊 监控告警
- 📱 低端机误报 200+/h
- 🔍 定位到 sched_latency.o
- 🐛 阈值硬编码 1ms

→

**开发修复**
- 📝 修改 BPF C 源码
- 🔄 阈值改为 Map 读取
- 🧪 本地验证通过
- 📋 提交修复版本 v1.3

→

**编译 & 签名**
- 🔨 clang 编译 .o
- 🔑 ECDSA 签名
- 📦 写入 .bpf_sig section
- ☁️ 上传至云端仓库

→

**灰度验证**
- 📡 推送 1% 设备
- ✅ 验签通过 → 热加载
- 📉 误报率降至 0
- ⏱ 观察 1 小时确认

→

**全量推送**
- 📤 推送全部目标设备
- 🔄 Bpfloader 卸载旧版
- 📥 加载新 .o 到内核
- 🎉 修复完成，零停机

### 📱 设备端热替换流程

📥 **接收新 .o** — TLS 下载 + SHA256 校验 → ✅ **验签** — .bpf_sig ECDSA 校验 → 🛑 **卸载旧程序** — detach + unpin + close → 📖 **加载新程序** — parse ELF + CO-RE fix → 📌 **Attach** — pin map + attach hook

- ⏱ 整个替换过程 < 100ms，用户无感知
- 🔒 签名不通过则拒绝加载，回退到旧版本
- 📊 替换结果上报云端，支持远程回滚

✅ sched_latency.o v1.3 运行中 — 误报率 0%，修复耗时 6 小时

# 动态加载/卸载流程

## 请求接入

通过系统属性触发 bpfloader 启动，创建 Socket 监听客户端的加载与卸载请求。

## ELF 签名验证

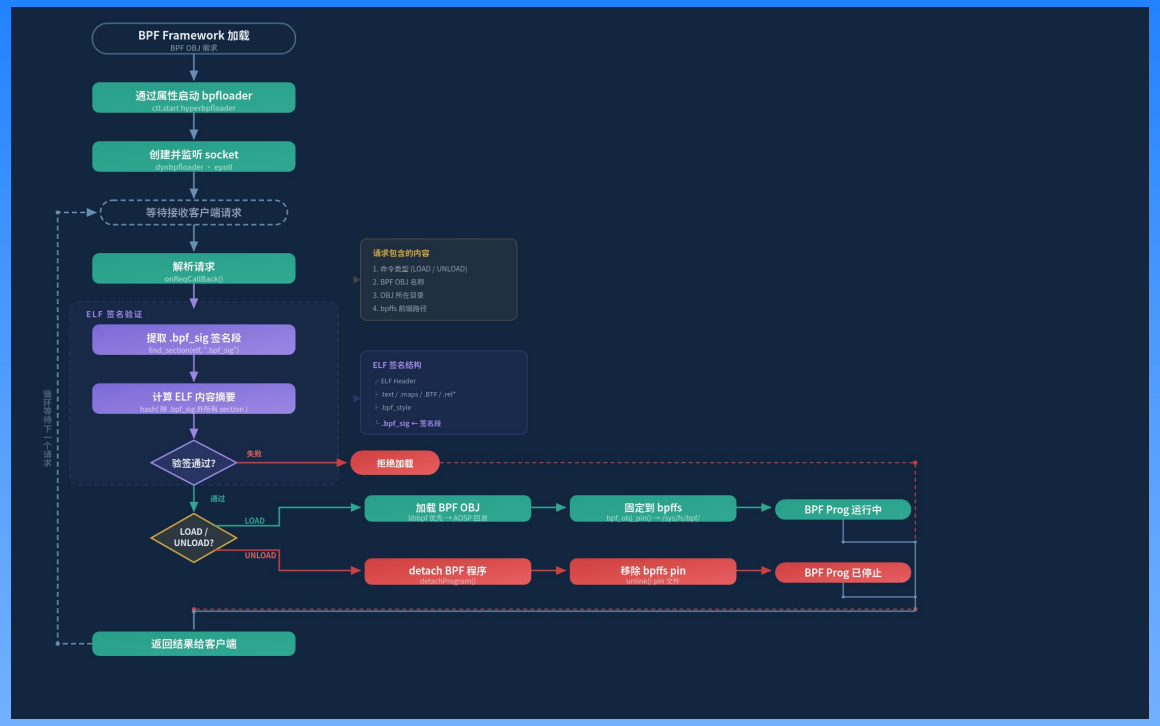
从 ELF 文件中提取签名数据，计算内容摘要后进行公钥验签；验签失败则拒绝加载。

## LOAD / UNLOAD 执行

加载时将 BPF OBJ 固定到 bpffs 并进入运行态；卸载时 detach 程序并移除 bpffs pin。

## 循环服务

操作完成后将结果返回客户端，随即回到监听状态，持续响应后续请求。



# AOSP BPF Loader

# Libbpf BPF Loader

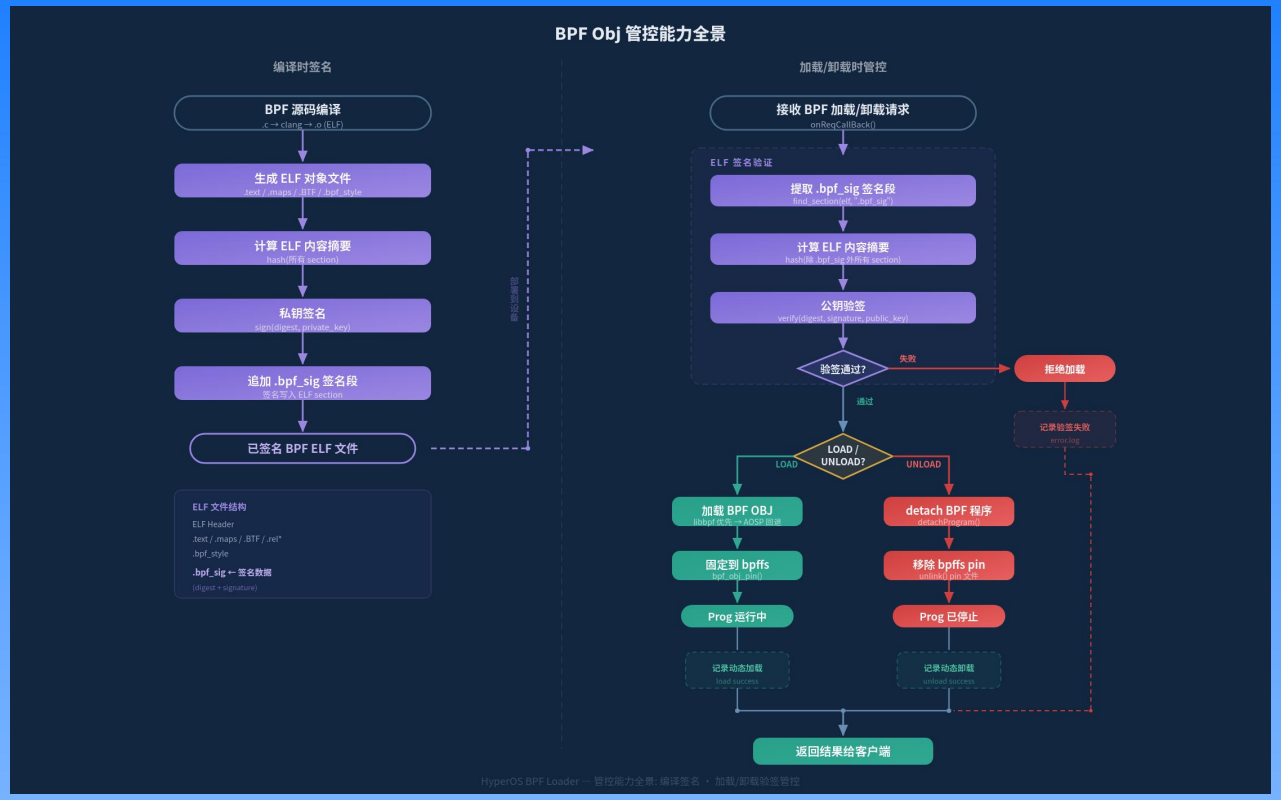
# 方案设计 - FW Service

## 加载/卸载时验签管控

接收加载/卸载请求后，从 ELF 文件中提取签名数据进行公钥验签；验签通过则执行操作，失败则拒绝加载。

## 日志审计

记录加载、卸载、验签失败等事件，为安全审计提供数据支撑。



# 演进思考

## 平台化与工具链

- **BPF 程序管理平台**
  规划统一的 BPF 程序版本管理与分发体系，实现签名校验、灰度推送、按机型适配等全生命周期管控
- **智能运维闭环**
  打通监控与数据平台，实现 BPF 程序状态可观测、异常可感知、问题可追溯的运维闭环
- **BPF 模板市场**
  提供常用场景的 BPF 程序模板，降低业务团队使用门槛，加速 BPF 技术在终端场景落地

## 社区与标准化

- **ELF 签名标准化探索**
  将 ELF 签名实践分享给社区，借鉴内核模块签名经验，共同探讨 BPF 签名规范方向
- **探讨 Android BPF 加载模型演进**
  结合动态加载实践，与社区交流移动端 BPF 加载模型的更多可能性
- **参与 BPF 社区标准制定**
  参与 BPF Foundation 等社区技术讨论，将移动端需求和实践纳入标准考量

# 感谢聆听

关于我们：

小米科技有限责任公司
内核技术部